



Devops testing - Best Practices

Address the Disconnect Between QA and Developers

QA is critical to the software release process. So why is it often thought about last when an organization embarks on a DevOps transformation initiative? There is a common misconception about QA's role in a DevOps environment: that with continuous integration and continuous delivery, all quality efforts can be automated too.

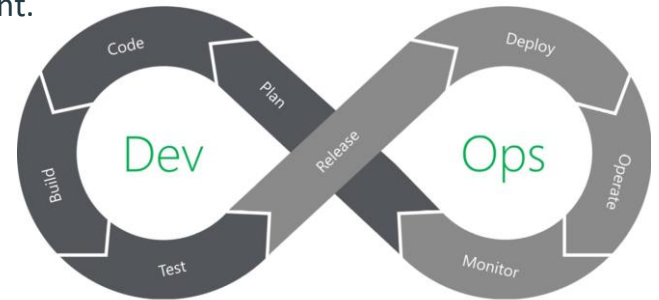
While it's true organizations are achieving a higher level of test automation than ever before, release readiness is a business decision – not a technical decision. And while automation can give you a high level of confidence that the release is ready for customers, good test results shouldn't be confused with release readiness. So how can you address the disconnect and ensure quality is embedded in DevOps processes? Include testers as part of your integrated delivery team.



Iteratively Adopt Practices and Tools for Each DevOps Stage

Rather than a rip-and-replace approach to tooling, it's possible – and even preferred – to iteratively adopt practices and tools for each DevOps stage. To begin, leaders should address the biggest problem areas first. This is a much more approachable method, as it allows teams to adopt tools and practices over time, as their processes mature and needs evolve.

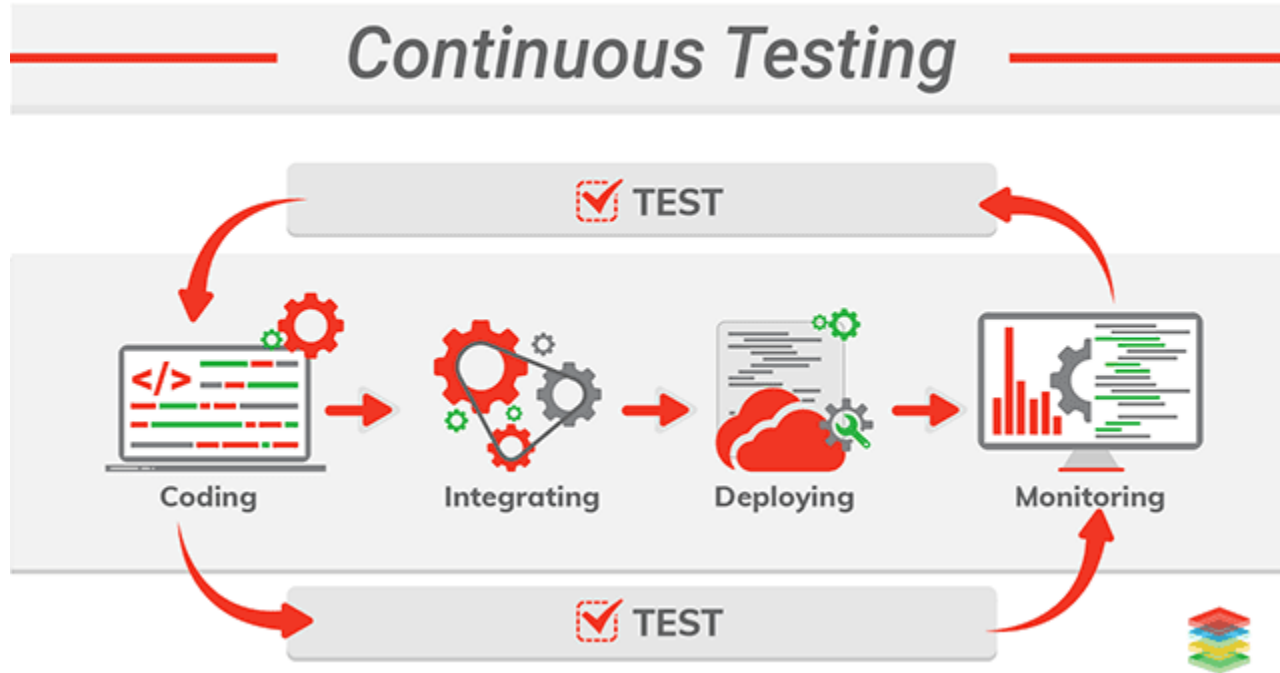
According to recent research from Gartner,** as an application leader, you should Drive your DevOps teams to develop their practices and toolkit iteratively by identifying the biggest constraint in their processes and selecting the quality tools that best help them optimize for that constraint.



Adopt Continuous Testing Practices

A recent study found that organizations with successful agile and DevOps initiatives do a number of things differently than their peers – including active efforts to transform their software testing practices into continuous testing practices. The study, which was led by Forrester analyst Diego Lo Giudice and commissioned by Tricentis, found that these organizations are adopting five core continuous testing practices: allocating proper testing budgets and focusing on improving skills; implementing continuous testing to meet the demands of release frequency; including automating end-to-end functional testing; integrating testers into cross-functional teams and practicing shift-left testing.

Adopt Continuous Testing Practices



Automate Testing Where it Makes Sense

It can be difficult to determine the ideal level of test automation your team should be striving for during the DevOps transition. But with releases happening significantly more frequently, the volume of testing must also increase significantly. That's where automated testing comes into play. According to recent research, successful DevOps shops are significantly more likely to have high levels of automation for key testing and QA processes (test case design, functional test automation, test data management, etc.).



Automate Testing Where it Makes Sense

However, teams shouldn't simply proceed with the goal of automating as much as possible. Instead, the goal should be to lean on the team's more tenured testers to develop a test automation strategy that maximizes resources and eliminates the need for testers to manually perform repetitive test runs. By eliminating rote tasks, automation allows testers to spend their time more strategically – both planning and executing automation strategies and performing the exploratory testing that cannot be replaced by automation

Focus on the Right Metrics

Determining the test cases that make the most sense to automate comes down to having good test data. Centralizing QA metrics to get a holistic view of all of your tests-across all of the delivery pipelines- will help you see how frequently tests are failing, how long they take to run, and how much time you are spending on environment setup and data creation.

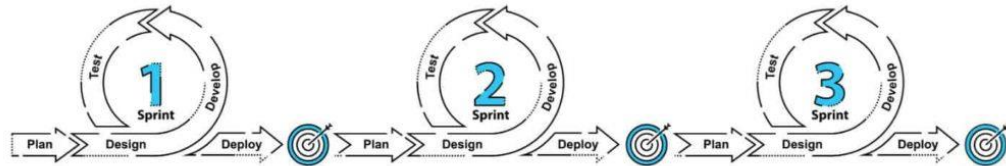
Once you've achieved a relatively high level of automation, the ability to quickly view and analyze metrics will become much more important. Access to the right metrics will allow you to gain insights that help you focus test automation strategies on the areas in which it can have the most significant impact for your business. So what are the right testing metrics? According to Forrester, agile and DevOps experts consistently rank risk-related metrics among their top 3 most valuable metrics. Organizations with mature agile and DevOps practices also tend to focus on contextual metrics such as requirements coverage, rather than simply counting the number of tests run or defects found.



Implement the Agile Methodology

A methodology can be defined as a set of best practices that aims to interpret and solve different problems within a given scope. In software development, a methodology is meant to help improve the development process. It is required to bring structure and help plan and manage the process.

Before DevOps was implemented, there were several methodologies that were proposed to structure development process: the Waterfall methodology, the V-model methodology, iterative and incremental methods, etc.. In these methodologies, specifications are frozen most of the time, allowing no room for changes throughout the development process. In addition, they often don't allow for frequent delivery. In the waterfall model, for instance, the product is designed, developed, tested and deployed during a long process, one step after another, sometimes leading to software that doesn't meet users' needs.



AGILE METHODOLOGY

Implement the Agile Methodology

Agile development enables and promotes continuous integration, which is in line with DevOps culture. With Agile and its implementation, software development is broken down into small tasks called stories. The iterative and incremental approach (Scrum for instance) gives room for continuous integration. Each time a story is finished, it is pushed in a shared repository where it is tested and validated first as a single feature and then as a part of a system. Once the product is validated, bugs and errors are reported to the backlog and will be part of the following iteration. The product owner designated at the beginning of the project will be part of the development process: UAT, requirements validation, backlog prioritization etc.

Agile helps approach a complex problem by breaking it down into more manageable pieces of features and modules. Iteration and a good control over time frame ensure that the product is developed according to the requirements and clients' needs which leads to a higher quality of the product which among DevOps' goals.

Implement Small Iterations for Better Control

Small changes are the correct way to implement continuous integration. Introducing a few changes at a time is what enables frequent integration. Once a small change is made, and integration has proved to be a success, the new release is delivered to the client. At this point, if bugs or errors occur it will be easier to detect their root causes. These bugs are often detected at early stage leading to fewer issues in the production environment. Validation and deployment can be done very fast as well leading to a reduced time to market and also to a frequent delivery.

Not like this...



...instead like this!



Document the Process

Documentation is also important for the deployment process. Deployment differs slightly from one application to another. These differences can include: configuration needed by the application (upload folder, database, etc.) and prerequisites in order to run the software (CPU, memory etc.). Standardized documents can help overcome this obstacle. The deployment process should be clear so that we are not wasting time reporting back and forth on deployment issues. We should have in mind that deployment in the DevOps context will happen frequently, so the documentation has to be as thorough as possible to speed the process

Document the Process

To conclude, there are four types of documentation that are essential while implementing DevOps:

- Source code documentation
- Design and requirements documentation
- How-to deploy
- Manuals for end users

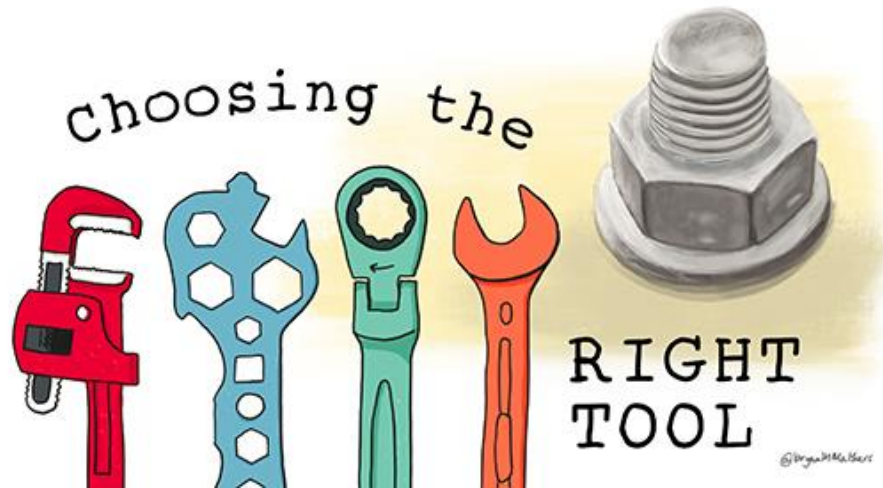


Choose the Right Tools

Every best practice has to be supported by tools.. These tools should be standardized among software developers and the operations team. This standardization is essential especially when the process involves more than one user.

Here are a few types of tools I recommend:

- Communication tools
- Continuous testing tools
- Version control tools
- Monitoring and tuning tools
- Agile tools



Be Reactive, Proactive and Open to Feedback

For high-quality software, it is important to be open to feedback and act on it as soon as it is given.

Feedback can be gathered through three different channels:

- From operations team to development team and vice versa
- From end-users to the development team
- From application logging: allow the software we develop to give us feedback

The operations team gives its feedback by reporting errors or bugs or by suggesting improvements. Several tuning and monitoring tools like Munin or Sentry One are now used to track issues, they have become so efficient that tracking problems really soon and identifying potential bottlenecks have become easier. Suggestion and bugs reporting help improve code quality and even help anticipate problems and find solutions before they affect production. Fixing a reported issue as soon as it occurs is more than a good practice. Efforts should be deployed to reduce time to fix a bug.

Be Reactive, Proactive and Open to Feedback

Applying lessons learned from previous challenges is one way to be more proactive. During the validation phase, situations that could possibly be causing problems can be triggered, the software has to pass these situations successfully in order to be eligible for a production deployment.

Another way to be more proactive is upfront communication: assume less and inform more. We have all suffered at least once from poor communication. The development team may sometimes engage in debugging for several days only to realize that it was due to changes in the production environment configuration.



Conclusion

Applying these practices is not as difficult as it may seem. Once the organization is set with the right tools and the culture is understood, it is easier to apply DevOps principles on a daily basis. Communication and collaboration via the right tools ensure that software development process is running smoothly.

Eventually, DevOps should come naturally, it is as natural as changes organization and business are undergoing all the time.



pytest



Unveiling the Basics

What is pytest?

- pytest stands as a robust testing framework tailored for Python, designed to streamline the testing process.
- At its core, pytest offers a clean, simple syntax and powerful features, enabling developers to write concise and maintainable tests.

Why use pytest for Python testing?

- pytest brings forth a multitude of advantages, notably its simplicity and flexibility, empowering developers to write tests efficiently.
- Its intuitive syntax and extensive plugin ecosystem foster productivity and enable tailored testing solutions.
- Furthermore, pytest seamlessly integrates with existing tools and frameworks, amplifying its utility in diverse Python projects.

Installing pytest

Requirements: Python: Verify that Python is installed on your system, with pytest supporting versions 3.6 and above.

Installation using a Virtual Environment:

- **Setup Virtual Environment:** Create a virtual environment using a tool like virtualenv or venv to isolate your project's dependencies.
- **Activate Virtual Environment:** Activate the virtual environment using the appropriate command for your operating system. `source venv/bin/activate`
- **Install pytest:** Within the activated virtual environment, use pip to install pytest: `pip install pytest`
- **Deactivate Virtual Environment (Optional):** Once pytest is installed, you can deactivate the virtual environment using the appropriate command. `deactivate`

Structure of a Basic Test

- A basic test in pytest is encapsulated within a Python function.
- The function name typically starts with test_, serving as a marker for pytest to identify it as a test.
- Within this function, you define assertions using Python's assert statement to validate specific behaviors of your code.

```
def add_numbers(x, y):  
    return x + y
```

```
def test_addition():  
    assert add_numbers(2, 3) == 5
```

Assertions in pytest

- Assertions play a crucial role in validating the behavior of code during testing. Let's delve into the world of assertions within pytest.
- In pytest, various common assertions are available to facilitate thorough testing. These assertions enable developers to verify expected outcomes and ensure code correctness with confidence.
- Utilizing assert statements effectively is paramount in writing robust tests. By strategically placing assert statements, developers can verify specific conditions, handle edge cases, and enhance test coverage. Understanding when and how to employ assert statements empowers developers to create comprehensive test suites that thoroughly validate their code's functionality.

Running Tests

- `pytest` provides a comprehensive set of command-line options that cater to diverse testing needs. These options offer fine-grained control over test execution parameters, enabling developers to tailor the testing environment to their preferences. For example, the `-v` option increases verbosity, providing detailed information about test outcomes and execution order. Similarly, the `--tb` option allows developers to specify the level of detail in traceback printing, facilitating efficient debugging. By leveraging these command-line options, developers can optimize their testing workflow and gain valuable insights into their code's behavior.
- `pytest` enables developers to target specific tests or subsets of tests with precision. By specifying test names, directories, or markers, developers can focus on particular areas of their codebase. This targeted approach not only accelerates test execution but also facilitates thorough testing of critical functionalities, ensuring code integrity and reliability.

Fixtures

- In pytest, fixtures are specialized functions designed to provide test data or set up necessary resources before test execution. They encapsulate common setup and teardown logic, promoting code reuse and enhancing test maintainability. Fixtures can be used to initialize database connections, set up mock objects, or prepare any other resources required for testing.
- Fixtures are utilized to ensure a consistent testing environment by executing setup actions before each test and teardown actions after each test completes. Before a test runs, the fixture prepares the necessary context or resources. Once the test finishes, the fixture cleans up any allocated resources or performs necessary teardown operations. This approach guarantees that each test operates independently and reliably, minimizing side effects and ensuring reproducibility across test runs. By leveraging fixtures, developers can streamline the testing process and focus on verifying the behavior of their code with confidence.

```
def calculate_rectangle_area(width, height):  
    return width * height
```

```
import pytest  
  
@pytest.fixture  
def rectangle_dimensions():  
    width = 5  
    height = 10  
    return width, height  
  
def test_calculate_area(rectangle_dimensions):  
    width, height = rectangle_dimensions  
    area = calculate_rectangle_area(width, height)  
    assert area == 50
```

Parameterized Tests

- Parameterized tests are a valuable feature in pytest, enabling the execution of the same test logic with multiple input values. Let's explore the concept of parameterization and its practical application in testing.
- Parameterization allows tests to be run with different sets of input data or configurations, making it easy to test a function's behavior across various scenarios.
- Parameterized tests in pytest involve decorating test functions with `@pytest.mark.parametrize` and specifying input parameters and their corresponding values. This allows developers to execute the same test logic with different inputs, ensuring comprehensive test coverage and enhancing test effectiveness.

```
def is_even(number):  
    return number % 2 == 0
```

```
import pytest  
  
@pytest.mark.parametrize("number, expected_result", [  
    (2, True),  
    (3, False),  
    (10, True),  
    (15, False)  
])  
def test_is_even(number, expected_result):  
    assert is_even(number) == expected_result
```

Using Pytest Plugins

Pytest plugins seamlessly integrate with the pytest framework, extending its functionality without requiring extensive modifications to existing test code. They provide hooks and APIs that enable developers to enhance test automation, improve test coverage, and streamline testing workflows with minimal effort. By simply installing and configuring plugins, developers can unlock additional features and optimizations tailored to their specific testing needs. Moreover, plugins often follow pytest's philosophy of simplicity and flexibility, ensuring compatibility with existing pytest conventions and workflows.

- **pytest-cov** for code coverage analysis
- **pytest-xdist** for parallel test execution
- **pytest-html** for generating HTML reports
- **pytest-django** for testing Django applications
- **pytest-flask** for testing Flask applications
- **pytest-mock** for simplified mocking and patching

Markers in Pytest

- Pytest allows developers to define custom markers using the `@pytest.mark` decorator. These markers serve as annotations that can be applied to individual test functions or entire test modules. For example, developers can create markers such as `@pytest.mark.smoke` for smoke tests or `@pytest.mark.performance` for performance tests. These markers help categorize tests based on their characteristics or requirements, facilitating targeted test execution.
- Once custom markers are defined, pytest provides a convenient way to run tests associated with specific markers using the `-m` option followed by the marker name. For instance, developers can execute only smoke tests by running `pytest -m smoke`. This selective approach to test execution allows developers to focus on specific test suites or scenarios, improving testing efficiency and enabling faster feedback loops. Additionally, pytest supports logical combinations of markers, enabling even more sophisticated test selection criteria.

Configuration with pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
markers =
    smoke: mark a test as a smoke test
    regression: mark a test as a regression test
    integration: mark a test as an integration test
    slow: mark a test as a slow-running test
addopts = -v -rsxX --tb=short
log_cli = true
log_cli_level = INFO
log_file = pytest.log
console_output_style = classic
```

pytest.ini enables developers to customize several aspects of test execution and behavior. For instance, developers can specify test paths to control which directories pytest searches for tests. They can define custom markers to categorize tests based on attributes like importance or type. Additionally, pytest.ini allows developers to configure verbosity levels for test output, specify output formats such as plain text or HTML reports, and set up logging options. By tailoring these configurations in pytest.ini, developers can establish consistent testing environments across projects, streamline testing workflows, and ensure adherence to project-specific testing conventions.

Test Coverage

- Test coverage measures the extent to which the source code is exercised by tests. It indicates the percentage of code lines, branches, or functions executed during testing compared to the total codebase.
- Various tools exist for generating coverage reports in Python projects, such as coverage.py, pytest-cov, and codecov. These tools analyze the codebase and provide detailed reports highlighting covered and uncovered code portions, helping developers gauge the thoroughness of their tests and identify areas for improvement.

```
===== 5 passed in 0.28s =====
(venv) [roman@MBP-de-Roman /Volumes/Projects/DanIT/devops3/Lessons/Testing-1]
| => pytest --cov=app
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: /Volumes/Projects/DanIT/devops3/Lessons/Testing-1
plugins: cov-5.0.0, flask-1.3.0
collected 5 items

test_app.py ..... [100%]

----- coverage: platform darwin, python 3.12.2-final-0 -----
Name      Stmts  Miss  Cover
-----
app.py      73     10   86%
TOTAL      73     10   86%
```

Advanced pytest Features

- pytest provides a rich set of hooks that allow developers to customize various aspects of the testing process. Hooks enable developers to intervene at different stages of test execution, such as before and after test runs, enabling advanced setup, teardown, and customization logic.
- pytest supports conditional test execution, allowing developers to run tests based on dynamic conditions or external factors. By using markers, command-line options, or custom logic within test functions, developers can selectively execute tests based on specific criteria, enhancing testing flexibility and efficiency.

```
import pytest
import sys

def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

@pytest.mark.skipif(sys.version_info < (3, 8), reason="Requires Python 3.8 or higher")
@pytest.mark.parametrize("number, expected_result", [
    (2, True),
    (3, True),
    (4, False),
    (5, True),
])

def test_is_prime(number, expected_result):
    assert is_prime(number) == expected_result
```

Setting Up a Flask Test Environment"

- When setting up a Flask test environment, developers typically create a separate configuration specifically tailored for testing purposes. This configuration often inherits from the main application configuration but can override settings as needed for testing scenarios. By configuring Flask to use testing settings, developers can ensure that tests are isolated and behave consistently.
- Additionally, the Flask test client is utilized to interact with the Flask application during testing. This client provides a simulated interface for making HTTP requests to application endpoints without the need for a full HTTP server. Developers can use the test client to send various types of requests, such as GET, POST, PUT, and DELETE, to different routes of the Flask application. They can then inspect the responses received from the application and make assertions to verify that the application behaves as expected under different conditions. This allows for thorough testing of the application's functionality in a controlled environment.

Writing Tests for Flask Routes

- When writing tests for Flask routes, developers typically focus on verifying the behavior of individual endpoints within the Flask application. This involves crafting tests to simulate various HTTP requests, such as GET, POST, PUT, and DELETE, to different routes of the application.
- During testing, developers scrutinize the responses received from these routes to ensure they meet the expected criteria. This includes checking status codes, response headers, and the content of the response body. Additionally, developers may validate the behavior of specific functionalities implemented in these routes, such as authentication, input validation, or data processing.

```
# app.py

from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def hello():
    return jsonify(message='Hello, Flask!'), 200
```

```
# test_app.py

import pytest
from app import app

@pytest.fixture
def client():
    with app.test_client() as client:
        yield client

def test_hello_route(client):
    response = client.get('/')
    assert response.status_code == 200
    assert response.json == {'message': 'Hello, Flask!'}
```

Mocking in Flask Tests

- Mocking is a vital technique in Flask testing, primarily because it allows developers to simulate the behavior of external dependencies or services without actually invoking them. This is particularly useful when testing Flask applications that interact with databases, APIs, or other external systems.

```
# app.py
import requests
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/weather')
def get_weather():
    try:
        response = requests.get('https://api.openweathermap.org/data/2.5/weather?q=Kyiv&appid=your_api_key')
        response.raise_for_status()
        weather_data = response.json()
        return jsonify(weather_data), 200
    except requests.RequestException:
        return jsonify({'error': 'Failed to fetch weather data'}), 500
```

```
# test_app.py
import pytest
from app import app
from unittest.mock import patch, Mock

@pytest.fixture
def client():
    with app.test_client() as client:
        yield client

@patch('app.requests.get')
def test_weather_success(mock_get, client):
    mock_response = Mock()
    mock_response.raise_for_status = Mock()
    mock_response.json.return_value = {'temperature': 20, 'conditions': 'Sunny'}
    mock_get.return_value = mock_response

    response = client.get('/weather')

    assert response.status_code == 200
    assert response.json == {'temperature': 20, 'conditions': 'Sunny'}

@patch('app.requests.get')
def test_weather_failure(mock_get, client):
    mock_get.side_effect = requests.RequestException()

    response = client.get('/weather')

    assert response.status_code == 500
    assert response.json == {'error': 'Failed to fetch weather data'}
```

Advanced Flask Testing Scenarios

- Authentication and authorization mechanisms are critical components of web applications to ensure secure access to resources. Testing these mechanisms involves simulating user authentication and verifying that protected routes are accessible only to authorized users. This may require mocking user login sessions, generating authentication tokens, or using custom authorization headers in test requests.
- Session management is essential for maintaining user state across requests in web applications. Testing session-related functionality involves managing session data, setting session cookies, and ensuring that session-dependent features work as expected. Developers may use techniques such as mocking session objects, manipulating session cookies, or utilizing Flask's testing utilities to simulate session behavior during tests.

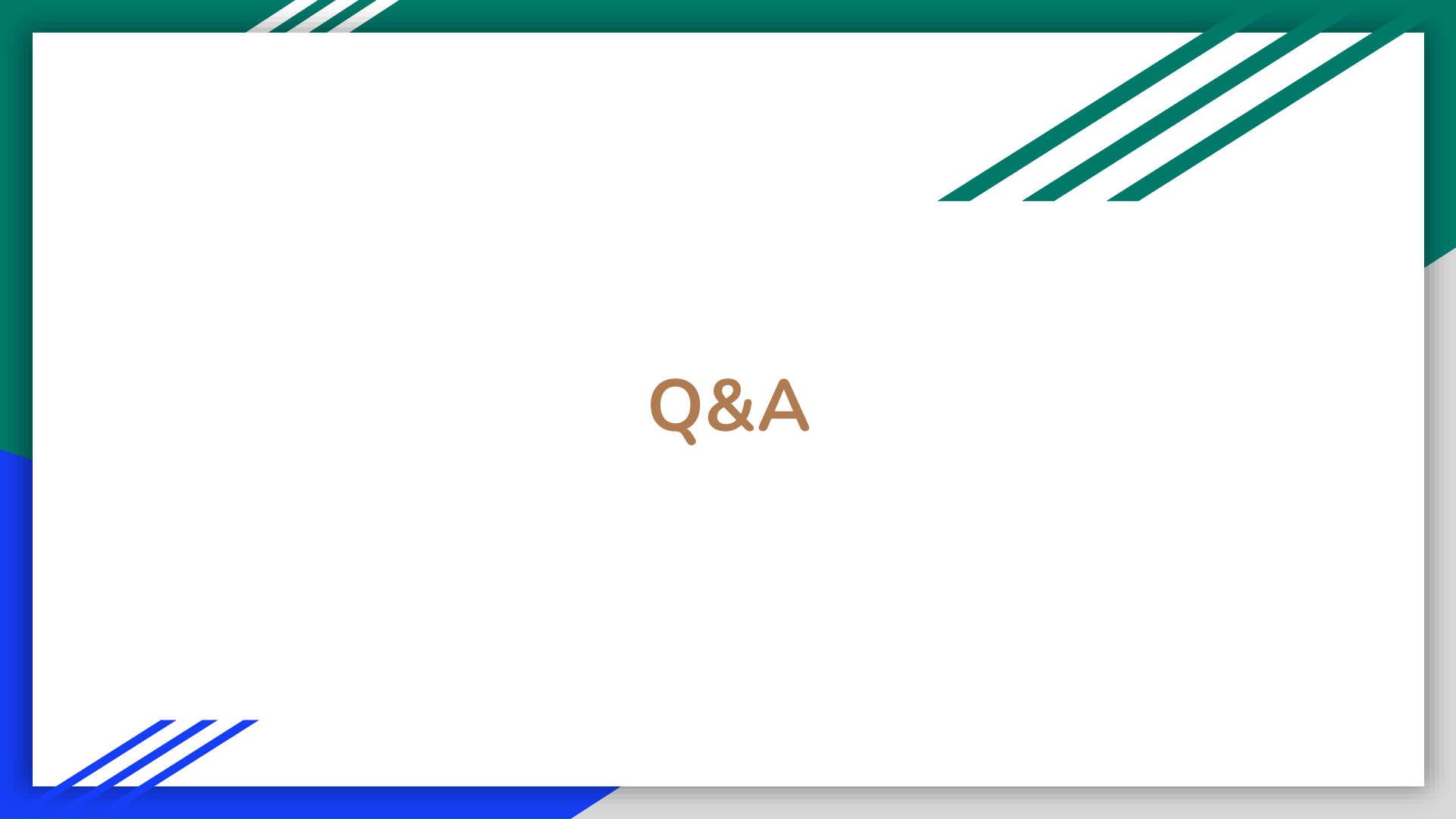
Best Practices in pytest

Organizing test files:

- Group tests into logical modules or packages based on functionality or components of the application.
- Use directories to organize tests hierarchically, mirroring the structure of the application.
- Keep test files separate from source code to maintain a clean project structure.

Naming conventions for tests:

- Use descriptive names that convey the purpose and behavior being tested.
- Prefix test functions with 'test_' to facilitate test discovery by pytest.
- Follow a consistent naming convention for test files and functions across the test suite.



Q&A